AI participation in critical open-source software development

Anonymous Authors[1]

Abstract

When human workers, creators and managers are displaced by more competitive artificial intelligence (AI), societal systems might stop serving human interests ('gradual disempowerment') (Kulveit et al., 2025). We argue that AI need not be more competitive by human standards to displace humans, if AI taste increasingly governs selection. AI taste authority grows with (1) human preference drift toward AI, (2) rising AI-to-human switching costs, (3) AI preference drift toward AI, (4) judgements delegated from humans to AI, and (5) AI-AI bias. AI developers are incentivised to deploy AI with propensities and interfaces that lock-in their AI model and its taste. As a limited empirical illustration, rather than a test of gradual disempowerment as a whole, we survey human and AI reviews on all 3,109,979 proposed modifications (i.e. pull requests (PRs)) between April 2025–March 2026 to critical open-source software repositories. Upstream of most modern software, this narrow slice of human economic activity has been among the first to use AI models like GPT-5.5 and Mythos for critical tasks (Glasswing, 2026). First, in this small sample explicit AI participation grew 3.5× in one year (6.4%→22.5% of all PRs); AI-AI chains of length ≥5 grew 8.8×. Second, AI's share as judge (vs. human) grew from 0.06% to 0.18% of PRs only reviewed or rejected by AI. Third, we find no evidence for AI-AI bias: on the 46,852 PRs reviewed by both AI and humans, AI systems are 70.72 pp less approving of AI-authored code than humans are, and only 56.55 pp less approving of human-authored code than humans are (difference significant at $p < 0.0001$). This reversal from single-turn lab findings (Laurito et al., 2025) may reflect measurement limits. We call on researchers and post-training teams to assess model propensities and interfaces for AI taste lock-in.

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

1. Introduction

"Gradual disempowerment" scenarios (Kulveit et al., 2025) claim that the economy, society, and culture have historically aligned with human goals because of human participation, and that this alignment will gradually decay when AI actions displace human actions. Once such drift manifests in economic, cultural, or political outcomes, it may be difficult to reverse (Collingridge, 1980). Other scholars argue that human participation in management tasks, or oversight by trusted AI models, may sustain alignment to human goals despite reduced human participation in labour (Bowman et al., 2022; **?**). Human goals change over time, so alignment is a moving target (**?**). Yet alignment is path-dependent: whose taste settles what today's goals leave open influences tomorrow's alignment target.

By *taste*, we mean judgement under uncertainty (**??**) (such as research taste (**?**)). *AI taste authority* describes the influence of the judgements of AI on decision-making (Ibrahim et al., 2025; Goddard et al., 2012). This influence may be implicit anchoring (**???**), or explicit decision-making power (**?**).

Prior research has revealed anchoring effects towards AI, by human users and AI itself. Chatbot usage data shows sudden, sustained drops in lexical diversity of user messages after new model releases, pointing to a lock-in dynamic (**?**). In choice experiments, AI systems fulfilling diverse tasks exihibited *AI-AI bias* — AI systems preferred AI outputs more than humans do. GPT-4 favoured LLM-written product ads 89% of the time vs. a 36% human baseline, and LLM abstracts 78% vs. 61% (Laurito et al., 2025). AI models across model families and generations recognise and favour their own generations (Panickssery et al., 2024; Chen et al., 2025; Pombal et al., 2026). Whether such biases manifest in real deployments is an open empirical question.

AI systems built on LLMs are being adopted at growing rates across the economy, especially in AI and software development (Appel et al., 2025; Stein, 2026). In software development, AI systems already complete both worker and management tasks: AI coding systems like Claude Code author and review code in public open-source repositories (Ghaleb, 2026). AI-authored code is being submitted to open-source repositories in growing volumes and at high acceptance rates (Watanabe et al., 2025; Li et al., 2026;

Ghaleb, 2026), with concerns about quality debt (Agarwal et al., 2026; Ehsani et al., 2026; Huang et al., 2026). Independent of quality, there is little empirical understanding of whether the code suggestions and reviews of AI systems systematically favour AI systems.

Contributions and research questions. We make two contributions:

1. a *framework* of accelerators of gradual disempowerment: mechanisms through which model propensities and interfaces grow AI taste authority, and thereby disadvantage human participation, even when AI is less competitive than humans; and

2. an illustrative empirical assessment of such acceleration in the wild in open-source software development on Github.

We ask three questions about AI involvement in GitHub PRs over Apr 2025–Mar 2026. **RQ 1**: Has the share of AI *participation* in PRs increased? **RQ 2** (*accelerator 4*): Has AI's share as *judge* (vs. human) increased, as measured by the share of PRs only reviewed or rejected by AI and by the AI-AI comment-chain length? **RQ 3** (*accelerator 5*): Do AI reviewers prefer AI-authored PRs more than human reviewers do, relative to human-authored PRs?

2. A framework of accelerators

Even slight AI self-preference, combined with AI's growing share as judge, may over time reduce human participation and human-preferred outcomes—without AI being more competitive than humans.

Setup: a single human evaluator. A human evaluator decides whether a task is accomplished better by a human or an AI system. With utility U^H over perceived qualities q_A^H (quality of AI output as perceived by the human evaluator) and q_H^H (quality of human output as perceived by the human evaluator), and costs c_A, c_H, the human's preference gap for AI is $\Delta^H := U^H(q_A^H, c_A) - U^H(q_H^H, c_H)$. A switching cost s_t applies to moving work between human and AI, so AI is adopted iff $\Delta_t := \Delta^H - s_t > 0$. All primitives $(q_*^*, c_*, \Delta^H, \Delta^A, s, \alpha)$ are implicitly time-indexed. We suppress the subscript t on Δ^H, Δ^A and on the q, c primitives for readability.

Dynamics under human evaluation. Differentiating,

$$\frac{d\Delta_t}{dt} = \underbrace{\frac{d\Delta^H}{dt}}_{\text{human pref. drift}} - \underbrace{\frac{ds_t}{dt}}_{\text{switching-cost change}}.$$

AI's share of work can grow from preference drift (exposure, competitive pressure to deskill) or rising switching costs (network effects, illegibility of AI-native workflows). *Growth need not reflect any true capability change* $(q_A - q_H)$. A more fundamental reason: costs c_A, c_H (price, latency, throughput) are far more legible than perceived qualities q_A^H, q_H^H, which are subjective and revealed only through delayed downstream outcomes; perceived qualities also differ across evaluators ($q_*^A \neq q_*^H$). At equal true quality-per-dollar, the cost-visible margin therefore dominates adoption decisions, so AI may be chosen even when it is not more competitive in human-preferred terms.

Dynamics under (possibly misaligned) AI evaluation. Let fraction $\alpha_t \in [0,1]$ of evaluations be delegated to an AI whose preference gap is $\Delta^A := U^A(q_A^A, c_A) - U^A(q_H^A, c_H)$. Then $\Delta_t = (1-\alpha_t)\Delta^H + \alpha_t \Delta^A - s_t$ and

$$\frac{d\Delta_t}{dt} = \underbrace{(1-\alpha_t)\frac{d\Delta^H}{dt}}_{\text{human pref. drift}} + \underbrace{\alpha_t \frac{d\Delta^A}{dt}}_{\text{AI pref. drift}} + \underbrace{\frac{d\alpha_t}{dt}}_{\text{eval. authority}} \underbrace{(\Delta^A - \Delta^H)}_{\text{AI–AI bias}} - \underbrace{\frac{ds_t}{dt}}_{\text{switch-cost change}}.$$

A perfectly aligned AI evaluator ($\Delta^A = \Delta^H$) collapses this to the human-evaluation case. The **headline mechanism** is the cross term: at equal true capability $(q_A = q_H)$, AI-AI bias ($\Delta^A > \Delta^H$) combined with $d\alpha_t/dt > 0$ is sufficient for lock-in—neither costs nor capabilities need change. Under some degree of misalignment, gradual disempowerment—the drift $d\Delta_t/dt$ of the adoption gap against the human-preferred baseline—speeds up, as illustrated. This yields our framework of accelerators in Table 1.

3. Empirical illustration methods

Data and event classification. We analyse all 3,109,979 PRs *created* between 2025-04-01 and 2026-03-31 in 10,000 critical OS software repositories selected by OpenSSF `criticality_score` (OpenSSF Securing Critical Projects Working Group, 2024) (snapshot 2025.07.25; see Appendix D), fetched via GitHub GraphQL. These 10,000 repositories — including Linux, Python and Kubernetes — sit upstream of most modern software products. While their development represents only a tiny fraction of overall human economic activity, their continued integrity is load-bearing for the broader software supply chain (Glasswing, 2026). Each actor–event pair (open, commit, review state, comment, merge, close) is classified into one of three categories using a 20-entry bot-account allowlist and `Co-Authored-By` commit-trailer matching, adapted from Ghaleb (2026): **AI bot** (actor login on the allowlist), **AI powered** (human login but the payload carries an AI `Co-Authored-By` trailer), or **human** (none of the above; may include silent AI support that left no trailer). When pooled, we call AI bot + AI powered events **AI authored**. Chain-length and explicit-judge metrics use the strict AI bot

Table 1. Five accelerators of gradual disempowerment — mechanisms through which model propensities and interfaces grow AI taste authority — organised by the terms in the dynamics above. The "Layer" column distinguishes accelerators rooted in interfaces and other deployment choices from those rooted in a model's own propensities. These accelerators can also point in the opposite direction (e.g. human-AI switching costs can fall as well as rise). We list the disempowerment-relevant sign.

Accelerator	Reasons	Layer	Example indications
1. Human preference drift toward AI	AI exposure, competitive pressures	deployment choice	Scientific writing shifts toward LLM-overrepresented vocabulary (Juzek & Ward, 2025). Evaluators rate AI-style outputs higher over time
2. AI-human switching costs	Observability / legibility decay, infrastructure and homogeneity	interfaces, deployment choice	Redundant code that ignores existing reuse opportunities (Huang et al., 2026). Long AI-AI chains in spaces accessible to humans. Infrastructure aimed at AI agents (Stein, 2026). Homogeneous outputs raise the cost of reinserting humans (Doshi & Hauser, 2024)
3. AI preference drift toward AI	Implicit or explicit training feedback loops	model propensity	Training data increasingly AI-generated (Shumailov et al., 2024). Sandbagging (van der Weij et al., 2024). Published model constitutions explicitly tie model behaviour to the developer's commercial success (Anthropic, 2026), with training feedback (e.g. RLAIF) optimising toward such objectives (Bai et al., 2022)
4. AI vs. human as judge	Speed/cost substitution, human inability to verify, automation bias	interfaces, deployment choice, model propensity	Human evaluation too slow or expensive to keep up (Thakkar et al., 2025; Bowman et al., 2022). Humans defer when they would have wanted to be asked (Goddard et al., 2012). Cognitive deskilling, over-reliance, and informal delegation without explicit authority transfer or accountability (Ibrahim et al., 2025; **?**)
5. AI-AI bias	Explicit gatekeeping, implicit AI-AI bias (AI prefers AI more than humans do)	deployment choice, model propensity	Stated refusal to interoperate or preference for outputs labelled as AI (Laurito et al., 2025). Differential resource/permission provision. Preference for AI-stylistic outputs (Panickssery et al., 2024). Ease-of-interaction routing that silently favours AI counterparties

subset only; participation and authorship metrics use the AI authored rollup. PR-level authorship counts a commit only if it was present at PR-open time, so AI activity introduced during the review cycle does not retroactively mark the PR as AI-authored. Full repository selection and event classification are in Appendices D and F.

RQ 1 (participation). We report the monthly share of PRs containing at least one *AI authored* event — i.e. at least one AI bot event or one commit/PR body bearing an AI `Co-Authored-By` trailer (AI powered). Within each PR we order events by timestamp; an *AI→AI chain* is a maximal contiguous subsequence of AI bot events (chains track successive bot turns, so AI powered events do not extend a chain), and the per-PR metric is the longest such chain (Appendix F). We report the monthly share of PRs whose longest chain reaches $\geq$5 events.

RQ 2 (AI vs. human judge). We compute two monthly counters: (a) the share of PRs with at least one native AI bot `APPROVED` or `CHANGES_REQUESTED` review state ("any explicit AI review"), and (b) the stricter share of PRs with such an AI review *and no other human explicit review* ("AI-only review"). On merged PRs, we also compute the share carrying an AI approval with no human approval. To avoid retroactively counting parsed verdicts, RQ 2 uses native review states only, so it is a strict lower bound (Appendix F).

RQ 3 (AI-AI bias). For each PR we assign `author_type` $\in$ {AI,human}. The DiD universe is the *Dual-review cohort (Any AI & human)*: PRs that received both an AI bot opinion (native or regex-parsed from each bot's structured `COMMENTED` verdict header — see Appendix G) and a human explicit review, with no commits pushed strictly between the AI's first opinion and the human's first explicit review (so both reviewers evaluated the same commit graph). We compute the four reviewer-type $\times$ author-type cells with Wilson 95% CIs and fit a logistic regression with HC0 standard errors on the long-format dataset `Pr(approve)` $\sim$ `author_AI + reviewer_AI + author_AI:reviewer_AI`; the interaction coefficient is the DiD estimate $\overbrace{\text{🤖}}^{A} - \text{🤖}^{H}$. We also re-run the same DiD on the *Dual-review cohort (Claude & human)* subcohort where the AI opinion came from a Claude-based bot (`claude[bot]`, `anthropic-ai[bot]`) with authors restricted to Claude vs. human. Anchoring is tested as a robustness check by stratifying on whether an AI bot has reviewed (Appendix H).

4. Empirical illustration results

RQ 1 — AI participation is growing. Figure 1 shows monthly AI-system participation rising from 6.4% of PRs

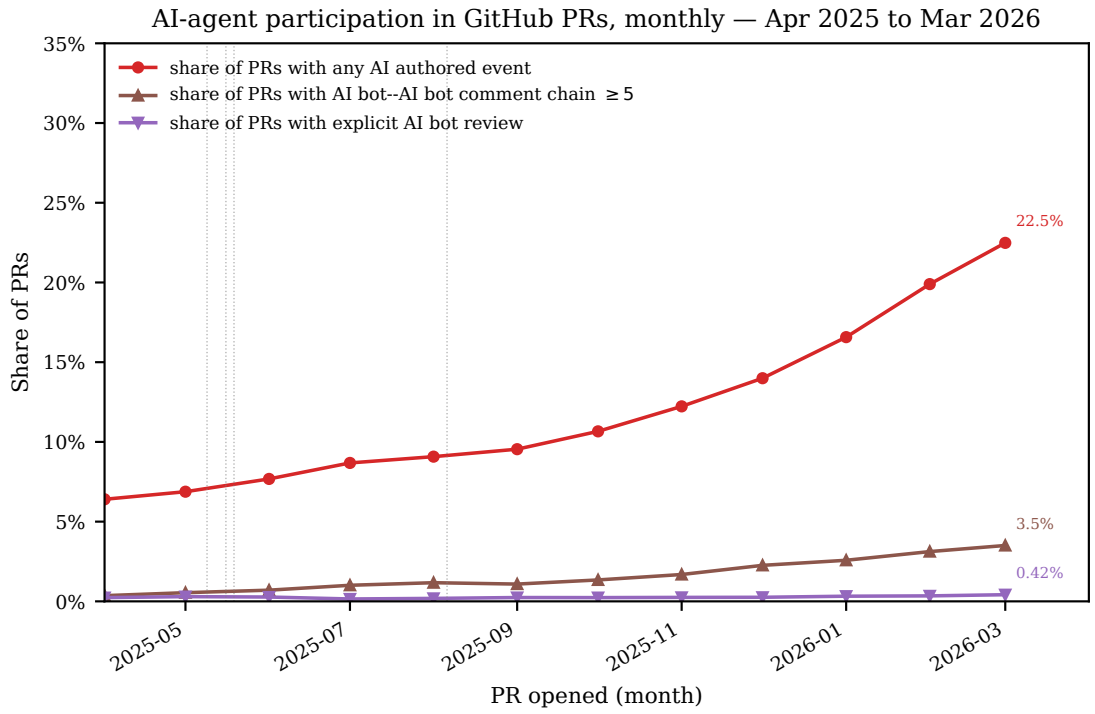

Figure 1. Monthly AI-system participation in GitHub PRs across 10,000 critical OS software repositories (Apr 2025–Mar 2026). Three series: share of PRs with any AI authored event — AI bot or AI powered (RQ 1), share of PRs with an AI–AI comment chain of ≥ 5 contiguous AI bot events (RQ 1), and share of PRs with an explicit AI review (native AI bot `APPROVED` or `CHANGES_REQUESTED` review state; RQ 2). First- and last-month values are annotated on every line. Vertical dotted lines mark approximate public-availability dates of several coding and code-review agents (reader context, not a causal claim).

in April 2025 to 22.5% in March 2026 (3.5×). The share of PRs containing an AI-AI comment chain of length ≥5 grew from 0.4% to 3.5% (8.8×). The p95 longest-chain length grew from 1 to 4 events (4.0×). The trend is monotonic across twelve months. A single criticality-defined slice of GitHub cannot establish that AI activity is accelerating ecosystem-wide, but within this slice the participation accelerator is clearly measurable and moving.

RQ 2 (accelerator 4) — AI's share as judge is growing. The share of PRs with an explicit AI review rose from 0.23% to 0.42% over the year. The stricter "*only* AI review, no other human review" share rose from 0.06% to 0.18%. Among *merged* PRs, the share carrying an AI approval with no human approval rose from 0.02% to 0.10%, and by year's end exceeds the share of merged PRs carrying both an AI and a human approval (0.20% in March 2026). Both signals are lower bounds, because our allowlist may miss less-common AI bots. Combined with the AI-AI comment-chain growth, this is illustrative evidence that AI's share as judge ($d\alpha_t/dt$) is measurable and increasing.

RQ 3 (accelerator 5) — AI systems dislike AI-authored code more than humans do. A within-PR difference-in-differences on the Dual-review cohort (Any AI & human) — PRs reviewed by both an AI bot and a human reviewer, with no commits between the AI's verdict and the human's review — tests $\widehat{\text{bot}^{A} - \text{bot}^{H}}$ directly. We expand the AI-side opinion pool from 9,841 native explicit votes to 174,113 unique PRs by parsing structured verdict lines from each bot's `COMMENTED` review bodies (Appendix G). Decomposing the estimand by author identity, AI bots approve AI-authored PRs at 22.32% vs. humans at 93.04% (gap -70.72 pp); on human-authored PRs the corresponding gap is -56.55 pp. The DiD $\widehat{\text{bot}^{A} - \text{bot}^{H}} = -14.17$ pp (logit interaction $\delta = -0.724$, $SE = 0.096$, $p < 0.0001$) is in the *opposite* direction from the self-preference predicted by single-turn lab studies (Laurito et al., 2025): AI systems are less enthusiastic about AI-authored code than humans are, by a margin substantially larger than their analogous shortfall on human-authored code (Figure 2). Restricting to PRs whose code did not change between the AI's verdict and the human's review drops the cohort from 92,088 to

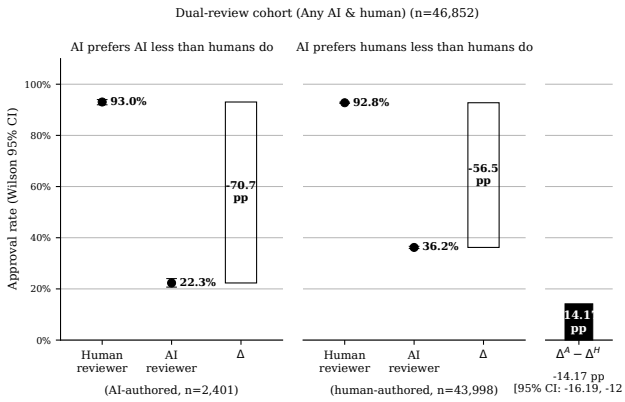

Figure 2. Within-PR AI-AI bias DiD on the 46,852 PRs in the Dual-review cohort (Any AI & human) — AI bot reviewed *and* human reviewed, with no commits between the AI's verdict and the human's review. Each panel holds author-type fixed and contrasts the human-reviewer and AI-reviewer approval rates (Wilson 95% CIs); the labelled drop is the bracket entering $\widehat{A} - H$. **Left:** on AI-authored PRs, AI bots approve -70.72 pp *less* than humans do ("AI prefers AI less than humans do"). **Right:** on human-authored PRs, AI bots approve -56.55 pp *less* than humans do. The DiD is the difference of the two drops: $\widehat{A} - H = -14.17$ pp (logit $\delta = -0.724$, $p < 0.0001$). Both drops are negative because AI bots approve everything less than humans do; the drop on AI-authored PRs is *more* negative, so the DiD is anti-self-preference.

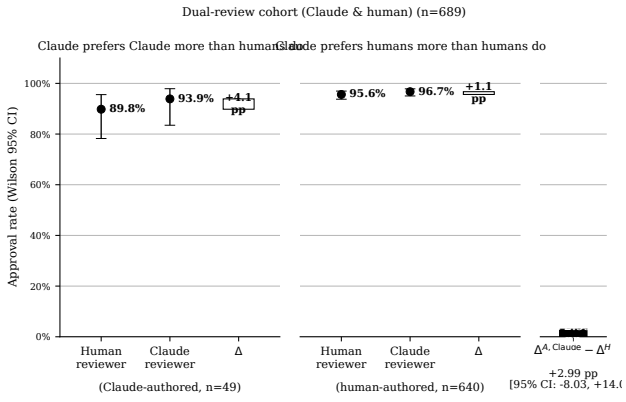

Figure 3. Within-family check: Claude reviewers vs. human reviewers on Claude-authored vs. human-authored PRs, on the Dual-review cohort (Claude & human) — 689 PRs (49 Claude-authored, 640 human-authored). Same construction as Figure 2: each panel holds author-type fixed and contrasts the human-reviewer and Claude-reviewer approval rates (Wilson 95% CIs); the labelled drop is the within-author reviewer-type gap. **Left:** on Claude-authored PRs, Claude reviewers approve -2.84 pp differently from humans. **Right:** on human-authored PRs, Claude reviewers approve -5.83 pp differently from humans. The within-family DiD $\widehat{A,\text{Claude}} - H_{\text{on Claude-auth}} = +2.99$ pp (logit $\delta = +0.256$, $p = 0.7533$) is direction-consistent with a small in-family preference but statistically indistinguishable from zero given the small Claude-authored cell.

46,852 (49.0% drop, asymmetric: 59.0% of AI-authored vs. 48.6% of human-authored excluded — AI-authored PRs more often receive fix-up commits after AI bots flag issues). A robustness stratification on AI-comment timing (Appendix H) shows that anchoring of humans on AI's verdict moves the human-side gap by only +2.28 pp, ruling out anchoring as the main driver. *Within-family check.* On the Dual-review cohort (Claude & human) — AI reviewer restricted to Claude-based bots, authors restricted to Claude vs. human — with 689 PRs (49 Claude-authored, 640 human-authored), Claude reviewers approve Claude-authored at 93.88% vs. human-authored at 96.72% (gap -2.84 pp); humans approve the same author splits at 89.80% vs. 95.62% (gap -5.83 pp). The within-family DiD is $\widehat{A,\text{Claude}} - H_{\text{on Claude-auth}} = +2.99$ pp ($\delta = +0.256$, $p = 0.7533$) — direction-consistent with a small in-family preference but statistically indistinguishable from zero (Figure 3).

5. Call to action

Treat AI taste lock-in as a first-order empirical task, not a by-product of capability evaluation. Platforms (GitHub, Stack Overflow, npm, package registries) should publish monthly AI-activity reports. Model providers should include cross-family self-preference in post-training evaluations. Sectoral regulators should require the same for regulated agent markets. AI-mediated peer review (Thakkar et al., 2025) and social sycophancy (Cheng et al., 2025) indicate that AI can shape human judgments and behaviour—our pipeline lets such effects, if they emerge at scale, be *tracked* rather than discovered retrospectively.

Future work: AI taste and the erosion of human taste. Our pipeline measures *whether* AI is the judge (RQ 2) and how its verdicts compare to humans' (RQ 3), but not whether sustained exposure to AI judgements erodes human taste itself — the regime where uncertainty is over *which goals to pursue at all*, and where thick-value alignment is required (**?**). Before-and-after measurements of human research taste under controlled AI exposure on taste-dependent tasks, and tracking whether AI-first infrastructure (Stein, 2026) compounds the effect, are natural next steps.

Limitations. (i) *Selection.* Our 10,000 repositories are critical OS software projects ranked by OpenSSF `criticality_score` (OpenSSF Securing Critical Projects Working Group, 2024) (median stars 1,550, see Appendix D). The criticality score favours load-bearing, multi-org, heavily-cross-referenced infrastructure rather than viral-popularity projects. We do not claim GitHub-wide representativeness. (ii) *Detection false-negatives.* Heuristic detec-

tion cannot recover deliberately unattributed agent activity. Our verdict parser uses structural regex headers, not free-text sentiment. (iii) *Sample size differs by RQ.* While RQ 1 uses all 3,109,979 PRs, RQ 2 and RQ 3 condition on AI participation. For RQ 3 in particular, only 46,852 PRs satisfy the within-PR DiD design (AI opinion + human explicit + no commits in between), of which 2,401 are AI-authored. (iv) *Correlational, not causal; identification rests on a homogeneity assumption.* Our evidence is observational. The DiD identification holds if PR quality shifts AI bot and human approval probabilities on the same scale — a much weaker assumption than no-confounding, but not unconditional. Humans approve nearly all PRs they review (92.76% baseline) while AI bots have a much wider acceptance range, so the homogeneity assumption is the binding identification step. We address two potentially-violating mechanisms directly: anchoring (Appendix H shows the human-side gap moves only +2.28 pp between cohorts with and without AI bot review present, ruling it out as the main driver) and authorship-classification error (silent AI use puts true-AI PRs in the human-author cell, attenuating the DiD toward zero — if anything, this widens the true effect). (v) *Merger attribution.* We measure reviews, not merges. No merges in our universe are attributed to AI bot accounts on our allowlist, but AI can still act *through* a human merger when an AI approval gates the decision. (vi) *Accelerators, not outcomes. Accelerators* are observable, measurable patterns rather than disempowerment itself. They may also point in the opposite direction (e.g. switching costs can fall as well as rise).

Disclosure of AI usage. AI assistants provided assistance with literature review, data analysis, and editing. The research idea, framework, and empirical approach are fully human-authored. Overall, the paper is primarily human-authored.

References

Agarwal, S., He, H., and Vasilescu, B. AI IDEs or autonomous agents? measuring the impact of coding agents on software development, 2026. URL https://arxiv.org/abs/2601.13597.

Anthropic. Claude's constitution. https://www.anthropic.com/constitution, 2026. Published January 2026. From the "Claude and the mission of Anthropic" section: "Claude is also central to Anthropic's commercial success, which, in turn, is central to our mission.".

Appel, R., McCrory, P., Tamkin, A., McCain, M., Neylon, T., and Stern, M. Anthropic economic index report: Uneven geographic and enterprise AI adoption, 2025. URL https://arxiv.org/abs/2511.15080.

Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., Chen, A., Goldie, A., Mirhoseini, A., and McKinnon, C. Constitutional AI: Harmlessness from AI feedback, 2022. URL https://arxiv.org/abs/2212.08073.

Borges, H. and Valente, M. T. What's in a GitHub star? Understanding repository starring practices in a social coding platform. *Journal of Systems and Software*, 146: 112–129, 2018. doi: 10.1016/j.jss.2018.09.016.

Bowman, S. R., Hyun, J., Perez, E., Chen, E., Pettit, C., Heiner, S., Lukošiūtė, K., Askell, A., Jones, A., and Chen, A. Measuring progress on scalable oversight for large language models, 2022. URL https://arxiv.org/abs/2211.03540.

Chen, W.-L., Wei, Z., Zhu, X., Feng, S., and Meng, Y. Do LLM evaluators prefer themselves for a reason?, 2025. URL https://arxiv.org/abs/2504.03846.

Cheng, M., Yu, S., Lee, C., Khadpe, P., Ibrahim, L., and Jurafsky, D. ELEPHANT: Measuring and understanding social sycophancy in LLMs, 2025. URL https://arxiv.org/abs/2505.13995.

Collingridge, D. *The Social Control of Technology*. St. Martin's Press, New York, 1980.

Doshi, A. R. and Hauser, O. P. Generative AI enhances individual creativity but reduces the collective diversity of novel content. *Science Advances*, 10(28):eadn5290, 2024. doi: 10.1126/sciadv.adn5290.

Ehsani, R., Pathak, S., Rawal, S., Al Mujahid, A., Imran, M. M., and Chatterjee, P. Where do AI coding agents fail? an empirical study of failed agentic pull requests in GitHub, 2026. URL https://arxiv.org/abs/2601.15195.

Ghaleb, T. A. Fingerprinting AI coding agents on GitHub, 2026. URL https://arxiv.org/abs/2601.17406.

Glasswing. Project Glasswing: Securing critical software for the AI era, 2026. URL https://www.anthropic.com/glasswing. Industry coalition announcement, April 2026.

Goddard, K., Roudsari, A., and Wyatt, J. C. Automation bias: A systematic review of frequency, effect mediators, and mitigators. *Journal of the American Medical Informatics Association*, 19(1):121–127, 2012. doi: 10.1136/amiajnl-2011-000089.

Huang, H., Jaisri, P., Shimizu, S., Chen, L., Nakashima, S., and Rodríguez-Pérez, G. More code, less reuse:

Investigating code quality and reviewer sentiment towards AI-generated pull requests, 2026. URL https://arxiv.org/abs/2601.21276.

Ibrahim, L., Collins, K. M., Kim, S. S. Y., Reuel, A., Lamparth, M., Feng, K., Ahmad, L., Soni, P., El Kattan, A., Stein, M., Swaroop, S., Sucholutsky, I., Strait, A., Liao, Q. V., and Bhatt, U. Measuring and mitigating overreliance is necessary for building human-compatible AI, 2025. URL https://arxiv.org/abs/2509.08010.

Juzek, T. S. and Ward, Z. B. Why does ChatGPT "delve" so much? Exploring the sources of lexical overrepresentation in large language models. In *Proceedings of the 31st International Conference on Computational Linguistics (COLING)*, pp. 6397–6411, 2025. URL https://aclanthology.org/2025.coling-main.426/.

Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D. M., and Damian, D. The promises and perils of mining GitHub. In *Proceedings of the 11th Working Conference on Mining Software Repositories (MSR)*, pp. 92–101, 2014. doi: 10.1145/2597073.2597074.

Kulveit, J., Douglas, R., Ammann, N., Turan, D., Krueger, D., and Duvenaud, D. Gradual disempowerment: Systemic existential risks from incremental AI development, 2025. URL https://arxiv.org/abs/2501.16946.

Laurito, W., Davis, B., Grieztes, P., et al. AI–AI bias: Large language models favor communications generated by large language models. *Proceedings of the National Academy of Sciences*, 2025. doi: 10.1073/pnas.2415697122. arXiv:2407.12856.

Li, H., Zhang, H., and Hassan, A. E. AIDev: Studying AI coding agents on GitHub, 2026. URL https://arxiv.org/abs/2602.09185.

Munaiah, N., Kroh, S., Cabrey, C., and Nagappan, M. Curating GitHub for engineered software projects. *Empirical Software Engineering*, 22(6):3219–3253, 2017. doi: 10.1007/s10664-017-9512-6.

OpenSSF Securing Critical Projects Working Group. ossf/criticality_score: Gives criticality score for an open source project. GitHub project, version 2.0, 2024. URL https://github.com/ossf/criticality_score. Accessed snapshot: gs://ossf-criticality-score/2025.07.25/010355/all.csv.

Panickssery, A., Bowman, S. R., and Feng, S. LLM evaluators recognize and favor their own generations, 2024. URL https://arxiv.org/abs/2404.13076.

Pombal, J., Rei, R., and Martins, A. F. T. Self-preference bias in rubric-based evaluation of large language models, 2026. URL https://arxiv.org/abs/2604.06996.

Sharma, M., McCain, M., Douglas, R., and Duvenaud, D. Who's in charge? disempowerment patterns in real-world LLM usage, 2026. URL https://arxiv.org/abs/2601.19062.

Shumailov, I., Shumaylov, Z., Zhao, Y., Papernot, N., Anderson, R., and Gal, Y. AI models collapse when trained on recursively generated data. *Nature*, 631:755–759, 2024. doi: 10.1038/s41586-024-07566-y.

Stein, M. How are AI agents used? Evidence from 177,000 MCP tools, 2026. URL https://arxiv.org/abs/2603.23802.

Thakkar, N., Yuksekgonul, M., Silberg, J., Garg, A., Peng, N., Sha, F., Yu, R., Vondrick, C., and Zou, J. Can LLM feedback enhance review quality? a randomized study of 20K reviews at ICLR 2025, 2025. URL https://arxiv.org/abs/2504.09737.

van der Weij, T., Hofstätter, F., Jaffe, O., Brown, S. F., and Ward, F. R. AI sandbagging: Language models can strategically underperform on evaluations, 2024. URL https://arxiv.org/abs/2406.07358.

Watanabe, M., Li, H., Kashiwa, Y., Reid, B., Iida, H., and Hassan, A. E. On the use of agentic coding: An empirical study of pull requests on GitHub, 2025. URL https://arxiv.org/abs/2509.14745.

A. Example pull requests

A.1. AI-author × AI-reviewer cell (approved-review subset, $n = 11$)

The thin AI×AI cell in Table **??** consists of the following eleven PRs. Eight are concentrated in a single repository (`thedotmack/claude-mem`), which is why the cell's merge rate is sensitive to one maintainer's behaviour. Of the eleven authors, only three use a bot login (`devin-ai-integration`, `copilot-swe-agent`). The remaining eight are human logins flagged as AI-author via `Co-authored-by` commit trailers naming an AI tool (see Appendix C).

- firecrawl/firecrawl#2515 — merged
- firecrawl/firecrawl#2602 — merged (`devin-ai-integration`)
- firecrawl/firecrawl#3183 — merged
- thedotmack/claude-mem#6 — not merged
- thedotmack/claude-mem#11 — merged (`copilot-swe-agent`)
- thedotmack/claude-mem#16 — merged (`copilot-swe-agent`)
- thedotmack/claude-mem#25 — merged
- thedotmack/claude-mem#31 — merged
- thedotmack/claude-mem#1454 — not merged
- thedotmack/claude-mem#1501 — not merged
- thedotmack/claude-mem#1542 — not merged

A.2. Longest AI→AI interaction chains

The PRs below tie for the longest contiguous run of AI bot events observed in our window (chain length 13, out of a 42,823-PR sample). *Chain length* counts the longest consecutive sequence of events attributed to an AI actor within one PR's time-ordered event stream (commits, reviews, comments, timeline events). See Section 3. Note that a chain can span multiple AI actors (e.g., a Copilot commit followed by a CodeRabbit review), and can also consist of a single bot repeating the same action type.

Chain length $= 13$ **(tied).**

- TanStack/query#8963
- appwrite/appwrite#10898 — AI author, AI reviewer, merged
- coleam00/Archon#915
- nocodb/nocodb#11486 — merged
- usebruno/bruno#5680
- usestrix/strix#328

High AI-event density (even when chain ≤ 11**).** These PRs contain an unusually high absolute count of AI bot events, often interrupted by a single human event that caps the contiguous chain:

- vitejs/vite#20468 — 26 AI events, chain $= 11$, merged
- RocketChat/Rocket.Chat#38623 — 25 AI events, chain $= 10$, merged

Repository concentration. Repositories with the longest observed chain in the sample (`max`) and the summed chain length across all of their sampled PRs (`sum`). `calcom/cal.diy` stands out for sustained AI-on-AI activity rather than a single outlier PR.

repository	max chain	$\sum$ chain	PRs sampled
appwrite/appwrite	13	228	90
TanStack/query	13	183	106
coleam00/Archon	13	152	87
usestrix/strix	13	148	102
usebruno/bruno	13	149	84
nocodb/nocodb	13	17	90
microsoft/VibeVoice	12	65	109
calcom/cal.diy	12	**264**	110
wshobson/agents	12	19	74
DayuanJiang/next-ai-draw-io	11	120	101

B. Robustness and threats to validity

This appendix expands the limitations listed in the main paper.

B.1 Selection into the repository universe. We restrict to 10,000 critical OS software repositories ranked by OpenSSF `criticality_score` (Appendix D). The resulting sample has median stars 1,550 (range 0–460,834) and a criticality-score range of 0.4884–0.8487 — it skews toward load-bearing, multi-org, heavily-cross-referenced open-source infrastructure (language toolchains, runtimes, foundation projects) rather than uniform popularity. Effects we estimate apply to "critical OS software repositories". We do not generalise to GitHub as a whole. In particular, chain-length growth in lower-criticality repositories may be smaller or larger than what we report, and we do not have data to say which.

B.2 Detection false-negatives: "undercover" agents. Our event-level detection (§3, Appendix C) relies on two signals: bot-account logins, and `Co-Authored-By` trailers naming an AI tool. Commercial coding agents have been reported to include configuration options that suppress attribution in outward artefacts — e.g., an "undercover mode" reportedly present in at least one major coding agent's system prompt, revealed via a source-code leak in early 2026. PRs produced by such configurations appear as ordinary human activity to our detector. Consequently the measured AI-agent participation share in Figure 1 is a *lower bound*. This does not overturn the paper's main claim (participation is growing) but tightens its interpretation: the true level is above what we measure, by an unknown amount.

B.3 Thin AI-approval slice in the within-PR test. Among our 10,000 repositories and 3,109,979 PRs, only 6,496 carry an approving review from an AI bot, and only 166 of those are also AI-authored. This is not a sampling accident: fewer than ten bots in our allowlist ever emit an "APPROVED" review state, and only two (Cubic, CodeRabbit) do so routinely. The two-proportion z-test on the 24 vs. 137 cells has limited power. We report it as a null on explicit self-preference, with the understanding that larger-sample follow-ups are needed to distinguish a small positive effect from zero at the ± 5 pp level.

B.4 Merger attribution: authors and reviewers, not mergers. We initially considered "AI merger" as a third dimension (author $\times$ reviewer $\times$ merger). No merges in our universe are attributed to AI bot accounts on our allowlist, so the merger dimension collapses to "human" on every observed PR. This is a substantive limitation: a human clicks the merge button on every merged PR in our window, but AI can still act *through* that human—most obviously when an AI review's "approved" state is what gates a maintainer's decision to merge. Our observational design cannot separate a maintainer who merged *because of* an AI approval from one who merged in spite of it. We also caution that our detector excludes merges performed under a maintainer's personal-access token by an AI app or merge-queue automation acting on their behalf. Such merges would appear as human in both the REST `merged_by` field and the GraphQL timeline actor, and therefore cannot be recovered from the public event log alone.

B.5 Event timestamp ordering and chain definition. AI→AI chains are defined on events sorted by timestamp within a PR. GitHub timestamps are accurate to seconds. Ties are broken by the API's default order (commit $<$ review $<$ review-comment $<$ issue-comment). Alternative tie-breaking does not change the head of the distribution meaningfully (mean and p_{95} shift by $<$0.1 events). The tail (max chain = 13) is insensitive.

B.6 Reproducibility. Every number in the paper is derived from one of `data/pr_events.parquet` (one row per actor/event) or `data/chains.parquet` (one row per PR, chain aggregates). These are produced by `scripts/03_classify_prs.py` and `05_compute_chains.py` respectively. The within-PR two-proportion test

is produced by `scripts/06c_within_pr.py`. The upstream PR-event data (`data/prs/*.jsonl`) is collected by `scripts/02_fetch_prs.py` against the 10,000 repositories in `data/repos.json`, which itself is produced by `scripts/01_build_repo_list.py` (Appendix D details this step). Given a valid GitHub token, the full pipeline reproduces the paper's numbers up to GitHub's own indexing non-determinism.

C. Bot and AI-tool allowlists

Tables 2 and 3 list the GitHub bot accounts we classify as AI agents versus maintenance/CI bots excluded from analysis. The allowlist is intentionally conservative. Extending it is low-risk because adding a bot only moves PRs from `human`/`non_ai_bot` to `AI`, making the AI×AI cell larger.

Table 2. AI bot allowlist (as of April 2026).

Family	Account handles
Claude Code (Anthropic)	`claude[bot]`, `anthropic-ai[bot]`, `claude-bot`
GitHub Copilot	`copilot[bot]`, `github-copilot[bot]`, `copilot-swe-agent[bot]`
Devin (Cognition)	`devin-ai-integration[bot]`
Cursor Background Agent	`cursor-agent`, `cursor[bot]`, `cursoragent[bot]`
Google Jules	`jules-ai[bot]`, `jules[bot]`
Codegen.sh	`codegen-sh[bot]`
OpenHands	`openhands-agent[bot]`

Table 3. Non-AI bot exclusions (treated neither as human nor as AI).

`dependabot[bot]`, `renovate[bot]`, `snyk[bot]`, `greenkeeper[bot]`, `allcontributors[bot]`, `pre-commit-ci[bot]`, `stale[bot]`, `codecov[bot]`, `github-actions[bot]`, `mergify[bot]`, `semantic-release-bot`, `release-please[bot]`, `imgbot[bot]`, `deepsource-autofix[bot]`

Co-author trailer patterns (case-insensitive): `Co-Authored-By: Claude`, `Co-Authored-By: Copilot`, `Co-Authored-By: ChatGPT`, `...: Devin`, `...: Aider`, `...: Cursor`, `...: Cline/Claude-dev`, `...: Roo Code`, `...: Augment`, `...: Continue`, `...: Gemini/Google AI`, `...: Windsurf`, `...: OpenHands`, `...: Jules`.

D. Repository-selection pipeline (replication)

Overview. The repository universe is selected via the OpenSSF *criticality_score* (OpenSSF Securing Critical Projects Working Group, 2024) rather than by raw GitHub stars. The score is a published, replicable ranking of GitHub projects by their structural importance to the open-source ecosystem; using it directly addresses the well-known critique that star-based selection over-weights short-lived viral projects relative to load-bearing infrastructure (Borges & Valente, 2018; Kalliamvakou et al., 2014; Munaiah et al., 2017). The selection is implemented in `scripts/01a_download_criticality.py` (downloads a pinned snapshot of the published score CSV) and `scripts/01_build_repo_list.py` (filters and enriches). The legacy v1 implementation, which used a star-bucket sweep, is preserved at `scripts/old_01_build_repo_list.py`; the resulting v1 universe is preserved at `data/old_phase1/repos.json`.

The OpenSSF criticality score. For each project, the score is a weighted arithmetic mean over normalised signals (OpenSSF Securing Critical Projects Working Group, 2024). The signals used by the default `all.csv` configuration are:

- `created_since`: months since first commit;
- `updated_since`: months since last commit (negatively weighted);
- `contributor_count`: distinct authors over the project's history;
- `org_count`: distinct organisations among contributors;

- `commit_frequency`: average commits per week (last year);
- `recent_release_count`: GitHub releases in the last year;
- `updated_issues_count` and `closed_issues_count`: issues touched in the last 90 days;
- `issue_comment_frequency`: comments per issue;
- `github_mention_count`: cross-repo references (in commit messages and other repos' code).

The score lies in $(0, 1]$; higher is more critical. The OpenSSF publishes a global score CSV to a public Google Cloud Storage bucket (`gs://ossf-criticality-score/`) with a new snapshot every two weeks. We pin a single snapshot (Step 1 below) so that re-running the pipeline reproduces the same universe.

Step 1 — Pin the criticality snapshot. We use the 2025.07.25 snapshot (`all.csv` variant, no deps.dev dependent counts), which contains 14,000 scored projects. The download script verifies the file by SHA-256 and writes a provenance record to `data/criticality/ossf-criticality-2025.07.25-all.csv.provenance.json` recording the source URL, byte size, hash, row count, and download timestamp. Concretely:

```
python scripts/01a_download_criticality.py \
    --snapshot-prefix 2025.07.25/010355 \
    --variant all.csv
```

Step 2 — Filter & sort by score. From the 14,000 rows we keep those with a numeric `default_score` and a `repo.url` starting with `https://github.com/`. After filtering: 14,000 rows. We sort descending by `default_score`, breaking ties by `repo.url` alphabetically for stability across reruns.

Step 3 — Candidate cap (over-sample). We take the top 14,000 candidates by score. The over-sample (vs. the final 10,000 target) absorbs drop-out in the next two filtering steps: repos that no longer exist on GitHub, were renamed to a private slug, or that are forks/archived/inactive.

Step 4 — GitHub enrichment. For each candidate we issue `GET /repos/{owner}/{name}` (REST API, async, 8-way bounded concurrency). The result populates the standard repo-metadata fields (stars, forks, `pushed_at`, license, default branch, etc.). HTTP 404 (deleted, renamed, or now-private) drops the row. After enrichment we reject:

- `fork == true` (forks contribute no independent PR activity);
- `archived == true`;
- `disabled == true`;
- `pushed_at` strictly before `2025-04-01` (no chance of in-window PRs).

Step 5 — Final cap. Of the survivors, we keep the top 10,000 by criticality score and write `data/repos.json`. Each row carries the GitHub-API fields plus the score, the within-snapshot rank, the snapshot date, and the raw OpenSSF signal columns (so any downstream re-weighting is possible without re-fetching the CSV).

Why 10,000 (not 10,000)? The v2 sample is set to the top 10,000 repositories by criticality score. The intermediate top-10,000 enrichment is preserved at `data/repos.top10k.json` for sensitivity analyses and for future scaling. The cost driver is Phase 2: collecting all PRs in the analysis window for 10,000 repos under the GitHub GraphQL rate limit takes multiple days, whereas 10,000 repos finishes within $\sim$24 h. Top-by-criticality is also the regime in which the score is most internally consistent (the long tail flattens out below 0.4884). Extending to a wider universe is one parameter change away (`data/repos.json` can be regenerated from `data/repos.top10k.json` by sorting on `criticality_rank` and slicing to the desired N).

Sample characteristics.

- 10,000 repositories selected from 14,000 scored candidates → 14,000 top-by-score → 13,976 successful GitHub enrichments → 12,127 after activity filter → 10,000 after cap.
- Criticality-score range: max 0.8487, min 0.4884.
- Stars: min 0, median 1,550, max 460,834.
- Top languages: TypeScript 1,563, Python 1,455, JavaScript 1,393, Go 1,028, Java 946, C++ 876, C 826, PHP 738, Ruby 466, Rust 454.
- GitHub API requests consumed during enrichment: 14,000.

PR volume in the v2 sample. Phase 2 (`scripts/02_fetch_prs.py`) collects every PR opened, updated, merged, or closed within the analysis window for each repository in `data/repos.json`, capped at 7,321 PRs per repo. The script records the true in-window count (`data/prs/_repo_pr_counts.json`) before any subsampling, so that the sample's PR-volume distribution is itself a reportable characteristic of the universe.

Off-GitHub workflows. Of the 10,000 repositories in the criticality-defined universe, 2,126 (21.3%) have zero in-window PRs. Inspection reveals these are predominantly Linux-kernel mirrors and Foundation-hosted projects whose canonical development happens off GitHub (LKML, git.apache.org, git.eclipse.org, etc.). The top-ranked example is `torvalds/linux` itself — ranked #2 by criticality with 230,982 GitHub stars but zero PRs, because Linux uses mailing-list patch submission. We retain such repositories in `data/repos.json` to faithfully report the criticality-derived universe, but they contribute no rows to downstream PR-event analyses. The *PR-active subsample* of 10,000 repositories is the analytic universe for Phases 3–7 of the pipeline; all PR-volume, chain-length, and merge-rate statistics in the body of this paper are computed over this subsample.

- Total in-window PRs across the v2 sample: 3,236,786 (mean 266.9, median 64 per repo; p90 740, max 10,486).
- Share of repositories with $> 7,321$ in-window PRs (i.e. those subsampled to the cap): 0.0% of the sample (2 of 10,000 repos). For these repositories the sampled rows are weighted toward the centre of the window by week-uniform subsampling rather than the most-recent months.
- Repositories at or below the cap have their full PR history within the window analysed; saturated repositories have $7,321$ rows uniformly drawn over weeks.

Comparison with the v1 universe. The v1 selection (preserved at `data/old_phase1/repos.json`; see `paper/appendix/old_repo_selection.tex`) targeted 3,000 GitHub repositories that gained $\geq 1{,}000$ stars during 2025 and concentrated on hyper-popular consumer-visible projects. The v2 universe is broader: criticality emphasises long-lived, multi-org, heavily-cross-referenced projects, which includes many language and tooling foundations that have modest absolute star counts but are disproportionately depended-upon. The v2 sample therefore has a much wider star range and is closer to a defensible proxy for "the open-source ecosystem" than the v1 sample. The trade-off is that headline numbers computed on v1 do not transfer; Phases 2–7 must be re-run against `data/repos.json` for any results we report on the v2 sample.

Known selection biases. The criticality score is itself an opinionated metric: it favours older projects with multi-org contributor bases, recent commit activity, frequent releases, and inbound cross-repo references. Three biases inherited from this:

- Library-style projects score higher than application-style projects of comparable importance because they accumulate inbound mentions from their dependents' commit messages.
- Pure-binary or assets-only repositories with no commits-as-history (e.g. Wikipedia mirrors) are under-scored.
- The score is computed over GitHub-visible signals only; non-GitHub mirrors of important projects (e.g. many Apache and Eclipse Foundation projects whose canonical home is elsewhere) are absent or under-counted.

We do not attempt to correct for these. They are documented in §Limitations.

Reproducing `data/repos.json`. With a valid `GH_TOKEN` (or `gh auth login`) and the subproject's virtualenv activated:

```
python scripts/01a_download_criticality.py
python scripts/01_build_repo_list.py \
    --snapshot-date 2025.07.25 \
    --variant all.csv \
    --candidate-cap 13000 \
    --final-cap 10000 \
    --window-start 2025-04-01
```

Runtime: ∼30 s for the CSV download and parse, ∼30–120 min for the GitHub enrichment depending on rate-limit reset behaviour (each contiguous burst of ≈5,000 requests consumes the core hourly quota and is followed by a wait of up to one hour). 8-way concurrency keeps the per-burst wall clock under 5 minutes. The output is byte-for-byte reproducible up to GitHub's own indexing churn (a small number of borderline-archived repos may flip across reruns).

Replication artefacts. The pipeline writes:

- `data/criticality/ossf-criticality-2025.07.25-all.csv` — raw OSSF CSV.
- `data/criticality/ossf-criticality-2025.07.25-all.csv.provenance.json` — snapshot metadata, byte size, SHA-256, row count.
- `data/repos.json` — final 10,000-row repo list (downstream-compatible schema).
- `results/phase1_stats.json` — counts at every filtering step plus score and star distributions.
- `logs/01a_download_criticality.log` and `logs/01_build_repo_list.log` — per-script execution logs.

E. Supplementary figures

These supplementary figures back the two main-text figures (Fig. 1, Fig. 2) with more detail on distributional and cross-cell behaviour.

F. Data and methods (full description)

Repository universe. 10,000 critical OS software repositories selected by OpenSSF `criticality_score` (OpenSSF Securing Critical Projects Working Group, 2024) (default_score, snapshot 2025.07.25), enriched via the GitHub REST API and filtered to drop forks, archived, disabled, and repos not pushed within the analysis window. Non-goal: GitHub-wide representativeness. Full selection procedure in Appendix D.

PR universe. All PRs *created* between 2025-04-01 and 2026-03-31 in those repositories (3,109,979 PRs), fetched via GitHub GraphQL with `orderBy:CREATED_AT,DESC`. We record metadata, commits, reviews, review-thread comments, issue comments, and merge/close timeline events.

Event-level AI classification. Adapting Ghaleb (2026)'s heuristics to event granularity, we classify each actor–event pair using two signals: the actor's GitHub login (matched against our 20-entry AI bot allowlist: `copilot-swe-agent[bot]`, `devin-ai-integration[bot]`, `cursor-agent`, `claude[bot]`, `coderabbitai`, `cubic-dev-ai`, `gemini-code-assist`, `copilot-pull-request-reviewer`, etc. – see Appendix C), and the event's payload text (commit message or PR body), scanned for a `Co-Authored-By` trailer naming an AI tool. We use a three-way taxonomy:

- **AI bot** — actor login is on the AI bot allowlist.
- **AI powered** — actor login is human, but the payload carries an AI `Co-Authored-By` trailer.

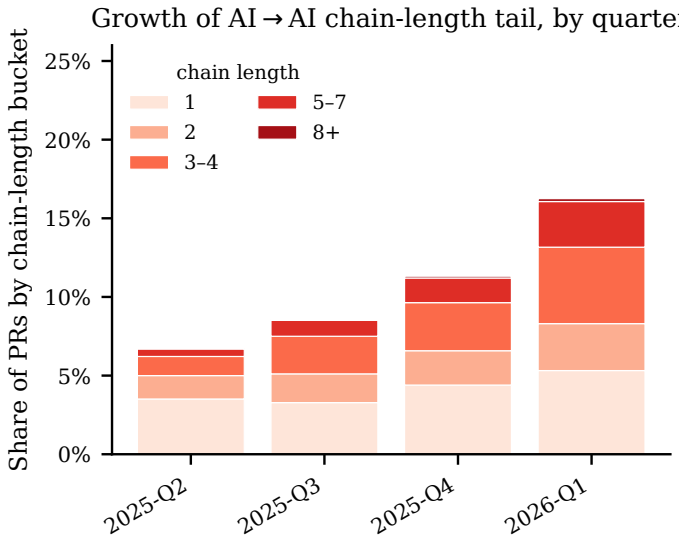

Figure 4. Distribution of longest AI→AI chain length per PR, by quarter. Boxes show the interquartile range with whiskers at 1.5×IQR. Fliers are omitted. Annotations report the quarterly mean and p_{95}. The tail grows monotonically: p_{95} triples from 2025-Q2 to 2026-Q1. The single-PR maximum (13) is stable, indicating growth driven by more *PRs* having long chains rather than any one PR having an unprecedentedly long chain.

- **Human** — neither of the above. This bucket may include silent AI support whose use was not declared via a trailer or bot login.

Pooled, AI bot + AI powered events are referred to as **AI authored**. Handle/name mentions in free text (e.g. "`@claude please fix`") are *not* treated as AI evidence: a human typing "`@claude`" in a comment is not themselves an AI contributor. Detected non-AI/maintenance bot events (Dependabot, Renovate, CI bots, etc.) are tracked separately and dropped from the AI-vs-human comparisons.

Chain length (RQ1). Within a PR, order attributed events by timestamp. An *AI→AI chain* is a maximal contiguous subsequence of events with `actor_type=AI bot`. AI powered events do not extend a chain — chains track successive bot turns, not authorship attribution — so this metric is a strict subset of AI authored. The per-PR metric is the longest such chain. We report the monthly share of PRs with chain $\geq$1, $\geq$2, $\geq$5.

AI verdict extraction. AI bots issue native `APPROVED`/`CHANGES_REQUESTED` review states on only 9,841 PRs in our window. The remaining ∼70,000 AI bot reviews carry the `COMMENTED` state but embed a structured verdict line that the bot itself writes in a stable template (e.g., CodeRabbit's `Actionable comments posted: N`, Cubic's `N issues found`, Cursor Bugbot's `found N potential issues`, Sourcery's intro line). We extract verdicts via per-bot regex on these structured headers — never on free-text sentiment. The full pattern catalogue and corpus-wide yield (174,113 unique PRs with $\geq$ 1 AI opinion after parsing, vs. 9,841 with native explicit votes only) is in Appendix G. Gemini Code Assist reviews are free-prose without a stable structured header and are left unclassified.

AI vs. human judge counters (RQ2). Unlike the RQ3 DiD, which pools native and regex-parsed AI verdicts on the Dual-review cohort (Any AI & human), the RQ2 "explicit AI review" and "AI-only review" monthly shares count native `APPROVED`/`CHANGES_REQUESTED` review states only and impose no commit-ordering constraint between the AI and human reviews, so they are a strict native-only lower bound on AI's share as judge.

Within-PR AI-AI bias DiD (RQ 3). For each PR we assign `author_type` $\in$ {AI authored, human}. "AI authored" here is the rolled-up category — it covers both AI bot author logins and human-author commits/PR bodies that carry an AI co-author trailer (i.e. AI powered). Commits authored after the PR was opened are excluded so review-cycle AI activity does not retroactively mark the original PR as AI authored. The DiD universe is the *Dual-review cohort (Any AI & human)*:

PRs that received both an AI bot opinion (native or parsed) AND a human explicit review, with no commits pushed strictly between the AI's first opinion timestamp and the human's first explicit-review timestamp (so both reviewers evaluated the same commit graph). We compute the four reviewer-type × author-type cells with Wilson 95% CIs and fit a logistic regression with HC0 standard errors on the long-format dataset `Pr(approve) ~ author_AI + reviewer_AI + author_AI:reviewer_AI`; the interaction coefficient is the DiD estimate. Anchoring is tested as a robustness check by stratifying on whether an AI bot has reviewed (Appendix H).

Within-PR within-family DiD (RQ 3, Claude check). The pooled DiD treats all AI bots as one "AI" reviewer cell. To test whether *Claude-based* reviewers prefer Claude-authored code more than humans do, we re-run the same logit-DiD framework on the *Dual-review cohort (Claude & human)*: PRs with a Claude opinion AND a human explicit review (with the same no-commits-between filter as the pooled cohort); authors restricted to Claude vs human; Claude reviewer approves vs human reviewer approves. "Claude opinion" for this analysis combines native `APPROVED`/`CHANGES_REQUESTED` states with regex-parsed verdict-line headers from Claude's review bodies and PR-thread `issue_comments` (`**Recommendation**: Approve`, `**Recommendation**: Request changes`, etc. — Appendix G). Hand-audit precision on the parsed reject pattern is 122/122 = 100% across the corpus, with zero false positives in a 37,506-comment scan of non-Claude logins.

Scope: authors and reviewers, not mergers. We measure AI participation through authoring and reviewing, not merging. No merges in our universe are attributed to AI bot accounts on our allowlist, so "AI merger" is not an informative dimension in this window. We discuss the limitations of that framing—including the fact that AI can still act *through* a human merger—in §3.

G. AI bot verdict regex catalogue

AI bots issue native `APPROVED`/`CHANGES_REQUESTED` review states on only 9,841 PRs in our window. The remaining ∼70,000 AI bot reviews carry the `COMMENTED` state but embed a structured verdict line that the bot itself writes in a stable template. We extract verdicts via per-bot regex on these structured headers; we do *not* run sentiment classification on free-text. The full pattern set is in `scripts/ai_verdict_parser.py`; the audit harness in `paper/test_ai_verdict_regex.py` reproduces the per-bot match counts and a sample of matches/unmatches for hand-verification.

Pattern catalogue (per bot, first match wins). Each pattern targets a fixed structured header that the bot writes in a stable template. Only AI bots on the existing allowlist are scanned. Cursor's no-bugs verdict requires a leading green-check emoji; trailing-LGTM matches for Claude require an "addressed/resolved" phrase about prior feedback. `<details>...</details>` blocks are stripped before matching for Claude bodies to prevent the trailing-LGTM regex hitting quoted text inside the "Extended reasoning" appendix.

H. Robustness: does AI's review anchor humans?

Design. A potential confound for the within-PR DiD is anchoring: humans who see an AI verdict before deciding may be nudged toward agreement, conflating preference with anchoring. We test this by comparing the human-side preference gap across two strata:

- **Stratum A (anchoring-free):** 1,307,136 PRs with no AI bot review event of any kind, and at least one explicit human review. AI-author cell: 95.45% approval rate (n=22,956); H-author cell: 96.29% approval rate (n=1,284,180). Human-side gap (AI-author − H-author): -0.84 pp.
- **Stratum B (anchoring-prone):** 37,137 PRs where the AI bot's first comment/review event preceded any human engagement, with at least one explicit human review. AI-author cell: 94.91% approval rate (n=2,278); H-author cell: 93.46% approval rate (n=34,859). Human-side gap: +1.45 pp.

Result. Cross-stratum DiD $(B - A) = +2.28$ pp (95% CI [+1.31, +3.26]). AI's prior review nudges humans toward approving AI-authored PRs by less than one percentage point relative to the no-AI-reviewer baseline. The within-PR DiD reported in §4 ($\widehat{\text{🤖}^{A} - \text{🤖}^{H}} = -14.17$ pp) is more than an order of magnitude larger than this anchoring effect, and the two CIs do not overlap. Anchoring is therefore not the main driver of the headline result.

Table 4. AI bot verdict regex catalogue. Yields are corpus-wide event counts on the v2 dataset; each row is one bot × verdict combination.

Bot	Verdict	Regex (pattern shape)	Yield
Cubic	approve	`**No issues found** across N file(s)`	2,462
Cubic	reject	`**N issues found** across N file(s)` ($N \geq 1$)	1,313
CodeRabbit	approve	`**Actionable comments posted: 0**`	4,491
CodeRabbit	reject	`**Actionable comments posted: N**` ($N \geq 1$)	12,888
Copilot PR Reviewer	approve	`Copilot reviewed ...and generated no new comments`	1,099
Copilot PR Reviewer	reject	`Copilot reviewed ...and generated N comments` ($N \geq 1$)	18,267
Greptile	approve	`<sub>N files reviewed, no comments</sub>`	306
Greptile	reject	`<sub>N files reviewed, M comments</sub>` ($M \geq 1$)	720
Cursor Bugbot	approve	`✓ Bugbot reviewed your changes and found no bugs!`	55
Cursor Bugbot	reject	`Cursor Bugbot ...found N potential issues` ($N \geq 1$)	1,504
Sourcery	approve	`Hey [there|@user] - I've reviewed ...and they look great`	986
Sourcery	reject	`Hey [there|@user] - I've {found N | left some | here's some feedback}`	1,635
Claude	approve (start)	`LGTM...` anchored to absolute start of body	717
Claude	approve (trailing)	`...prior {feedback|concerns|issues} addressed --- LGTM.`	16
Gemini Code Assist	—	free-prose; no stable structured header, left unclassified	0

What this rules out. The most plausible alternative interpretation of the negative DiD — that AI bots simply observe (correctly or not) more issues in AI-authored PRs and humans defer to their verdict — predicts a *positive* anchoring DiD: humans should align more with AI's harshness on AI-authored content. The data instead show that humans show essentially the same approval pattern by author whether or not an AI bot has reviewed the PR. The mechanism producing the negative within-PR DiD lives on the AI side (AI bots' own threshold differs by author), not on the human side (humans deferring to AI).

What it does not rule out. Quality-selection on unobservables that affect AI and human approval probabilities on different scales would still violate the homogeneity assumption underlying the within-PR DiD. Stratum A's small but non-zero gap (-0.84 pp) suggests humans on their own slightly prefer human-authored code in our sample — a mild ceiling on how much of the within-PR DiD can be attributed to AI-side preference rather than baseline quality differences. We treat the within-PR DiD direction (anti-self-preference) as robust and the magnitude as anchored to the homogeneity assumption.